Check for updates

Limits — A System for UNIX† Resource Administration

Andrew Bettison
Frank Adcock
Peter Chubb
Andrew Gollan
Chris Maltby
Neil Russell

Softway Pty Ltd
79 Myrtle St, Chippendale NSW 2008
Australia

ABSTRACT

The UNIX operating system, despite its emergence as a standard for supercomputer systems, lacks effective support for multiuser resource administration. The design and implementation of a decentralised resource administration system uniformly realisable across the wide variety of UNIX dialects presents a number of problems. Among these problems are potential violations of the UNIX design philosophy, preservation of the user process environment and adherence to industry standards.

A research project to design and develop a UNIX resource administration system built upon the "Fair Share" scheduler, Share, *has been undertaken at Softway with the support of the Australian Government. This project has yielded a resource administration and control system called* Limits *and further research is being undertaken on its extension to networks supporting shared resources.*

INTRODUCTION

As multi-user computer systems grow in power and complexity the need for effective resource control software becomes paramount. The supercomputer industry is adopting UNIX as its *de facto* standard operating system. Unfortunately, existing versions of UNIX fall short in the area of multi-user resource administration.

Any ideal operating system environment that supports a large population of concurrent users must provide its users with two things: security of data and protection from the actions of other users. A well-managed UNIX site ostensibly fulfils the first of these requirements but only partially fulfils the second. The administrator of a multi-user system often needs to divide its resources explicitly between groups of users in proportions corresponding to their respective claims to the system. On systems with a large user population there are a lot of routine, user-related administrative requests which would be dealt with most effectively by giving appropriate users the ability to service them. The UNIX system does not currently provide administrators with any straightforward ways to accomplish these things.

† UNIX is a trademark of AT&T Bell Laboratories.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.
© 1989 ACM 089791-341-8/89/0011/0686 $1.50

The *Limits* system is an extension of the UNIX operating system designed to address these shortcomings directly. *Limits* allows distribution of limited administrative power to nominated users by providing them with the means for controlling the allocation of users and resources within their own groups. *Limits* may be used to control users' and groups' consumption of standard resources such as hard disk, memory, terminal connect-time and printer pages, and may easily be customised to deal with others. By incorporating *Share* [1, 2, 3], (the "Fair Share" scheduler developed at the University of Sydney), as its CPU scheduler, *Limits* allows long-term, equitable sharing of the CPU amongst users and groups of users.

ADMINISTERING A TYPICAL UNIX SYSTEM

The responsibilities of the system administrator of a multi-user UNIX installation include:

- maintenance of continuous operation of the system by carrying out routine tasks such as system configuration, file system backups and system crash recovery;
- enforcement of an adequate level of system security to protect the system and its users from unwanted intrusion or misuse;
- prevention of poor system performance caused by problems such as excessively fragmented file systems, noisy modem inputs or malfunctioning daemon processes;
- maintenance of a sufficient level of unused space on all file systems to prevent users from running out of file space during the course of their work;
- general monitoring of resource demand and consumption to determine the need for additional equipment when appropriate;
- support for the user community by creating and maintaining user accounts and attending to administrative requests from users.

The latter three responsibilities demand significant effort on the part of a system administrator due to the lack of adequate administration software. The two particular areas of weakness are user administration and resource control.

User Administration

Sites with a large user population generally require more than one administrator to handle the high volume of administrative requests which naturally arise from a large and diverse community of users. In order for these system administrators to carry out their tasks, they must possess privileges superior to any normal user.

UNIX distinguishes between only two types of user: the normal user and the *superuser*. A process run by a normal user is able to invoke only a subset of the operating system's services and is subject to permission checking whenever it accesses a file. A process run by a superuser is able to invoke any function of the operating system and is allowed unchecked access to any file. Essentially, a superuser may be regarded as exempt from any forms of security or safety restrictions. On a secure system, users may only become superusers if they possess the superuser password.

Since a superuser is not subject to any restrictions it is easy for an unwary or inexperienced superuser to compromise the proper operation of the system. It is also easy for a superuser to inspect and alter other users' files, deliberately interfere with the processes of other users and interfere with any other aspect of the system's operation. It is often inappropriate to give full superuser status to all administrators, particularly those whose only responsibility is to provide routine support for users.

An obvious remedy for this situation would be a system which distinguished between different types of user privileges. An administrator would then be able to grant to selected users only those privileges necessary for dealing with routine user-related requests. As a result, administrative power would not be an all-or-nothing option, and an administrator could transfer a good deal of their responsibilities to more appropriate individuals. This basically amounts to a scheme for decentralising the user-related aspects of system administration.

Decentralisation of administrative power is not an established feature of typical UNIX systems. One way to compensate for this is to write *setuid* programs which perform most of the user-related administrative tasks. The privilege of invocation of such programs may be restricted to a specific set of otherwise normal users, thereby giving those users only as much administrative power as necessary. Unfortunately there is no widely available, standard suite of such programs. The need for individual development of such programs is a source of potential security holes, as many system administrators do not have sufficient experience in the complex issues involved.

Another problem faced by administrators who deal with many users is the lack of a standard facility for organising users into groups. It is often more appropriate to place resource constraints on a group of users rather than on each individual user in the group. Also, many administrative actions are concerned with a group of users, and require a simple method for specifying that the programs involved be applied only to users within that group. A system-wide regime for flexibly organising users into administrative groups would yield a simple and general means for addressing subsets of users by applying criteria based on their membership to one or more groups.

Resource Control

The fundamental objective of any operating system is effective resource management. The particular problem for multi-user operating systems like UNIX is to give many users apparently concurrent access to each resource without interfering with each other. A secondary problem is to prevent users from consuming more resources than they are entitled to.

In order to divide a physically continuous resource among several consumers it is necessary to present each consumer with an abstracted form of the resource. The primary resource abstractions provided by UNIX to its users are [4]:

- CPU time in the form of execution of processes;
- system memory in the form of process address space;
- hard disk storage in the form of files;
- user I/O in the form of terminals, network services, tape and floppy disk drives, printers, plotters and assorted peripheral devices.

These abstractions only partially serve to protect users from the effects of each other's actions. Although UNIX prevents normal users from interfering directly with the processes, memory and files of other users, there is no protection from the indirect effects of unregulated contention for resources. The only way to insulate users from each other completely is to prevent users from exceeding a reasonable level of consumption. This would mean that, although users would still be subject to limits on their consumption of resources, those limits would be predictable and amenable to negotiation.

Typical UNIX systems currently provide no general method for measuring (and optionally limiting) the consumption of available resources by users or groups of users. In particular, all available CPU schedulers are *process-level*, not *user-level* schedulers; that is, users are able to increase their current share of the CPU simply by running more concurrent processes. There are a few widely-used mechanisms for limiting the amount of disk space which a user's files may occupy, but none are sufficiently general to allow a limit to be placed on a group of users. Finally, UNIX currently provides no general platform for the development of customised forms of resource control.

THE DESIGN OF *LIMITS*

Philosophy

The UNIX operating system was originally designed and used to support small groups of co-operative research staff on small computers. There was no need to force compliance to strict security or resource constraints [5]. As UNIX evolved to support larger communities of competing users its security and resource abstractions were developed and tightened somewhat.

The development of UNIX has traditionally been characterised by a philosophy of allowing all users maximum possible freedom of action without compromising the security of other users. The addition of mechanisms for placing constraints on the freedom of users seems contradictory to this philosophy. One may adopt the view that a more mature philosophy of UNIX should be to provide users with maximum freedom of action without compromising the security *or system entitlement* of other users. In this case, the implementation of optional resource constraints for users is an unavoidable consequence of the definition of a user's *entitlement* to a part of the system.

Design Objectives

The overall objective of the *Limits* system is to provide the tools necessary for administering the resources of a system which supports many concurrent users. This involves addressing the areas in which UNIX is lacking, namely resource control and decentralisation of administrative power.

The general design objectives of *Limits* are:

- to provide a way of measuring and controlling users' consumption of existing UNIX resources such as CPU time, disk space, memory, printer pages and terminal connect time;
- to allow such resource control to be applied to all users and groups of users in a fully general manner;
- to allow extension and customisation of such resource control to accommodate a diverse range of special resources and administrative needs;
- to provide tools for decentralised resource administration which give sub-administrators control over only the resources available to a group of users;
- to be sufficiently flexible to allow an administration to

implement whatever policies it wishes;
- to avoid any serious alteration of the functional environment of UNIX as perceived by users and existing software systems, and in particular preserve accordance with relevant standards;
- to provide a platform for further development of more sophisticated forms of resource control and user administration.

User Hierarchy

The design of *Limits* approaches these objectives by establishing a method of organising users into a hierarchy of groups. A multi-level, hierarchical structure is appropriate for the kinds of organisational structures which exist in most establishments. A user is subject simultaneously to the resource constraints of all the groups to which they belong. This makes it possible to control resource allocation at many levels to reflect user, project and departmental entitlements.

Resource Models

Many resource control and accounting systems in the past have been based on a *fixed-budget model* charging scheme [6, 7, 8]. In a fixed-budget model, each user is given a fixed allowance of all machine resources. As resources are used, amounts calculated from the relative costs of the resources are deducted from users' allowances. It is often possible for users to choose higher or lower priorities for their jobs, thereby incurring lower or higher charges respectively for the resources they consume. When a user exhausts their allowance they are no longer able to use any resources until the system administrator advances their allowance.

A fixed-budget method of resource control has several limitations:
- it does not encourage users to spread their work evenly over time, and hence is of no use in reducing peak demand;
- it is possible for many users to exhaust their budgets while there are still plenty of some resources available. In other words, the fixed-budget model does not allow for the *renewable* nature of some resources;
- it does not help reduce the effect of a high-demand user causing poor response for other concurrent users.

Limits adopts different schemes for managing different resources. In particular, *Limits* makes a distinction between two classes of resource: *renewable* and *fixed*.
- A renewable resource is one which is in continuous supply such as printer pages, network packets or CPU seconds. Renewable resources may only be consumed, and, once consumed, cannot be reclaimed. *Limits* manages renewable resources using a usage and decay model with an optional limit.
- A fixed resource is one which is permanently present in a fixed amount, such as disk space and system memory. Fixed resources may be both consumed *and* relinquished. *Limits* manages fixed resources with a usage and limit model.

An example of how *Limits* manages renewable resources is its handling of printer pages. Each user has a current printer page usage variable, which is increased as they consume printer pages. When their usage reaches their printer page limit, they are prevented from using any more printer pages. At regular intervals their usage is decremented by a fixed amount, but it will never fall below zero.

An example of how *Limits* handles fixed resources is its memory limit scheme. Each user has a memory usage variable, which is equal to the amount of memory space which their processes currently occupy. Each user also has a memory limit, above which their net memory usage is not permitted to rise. Each user also has a per-process memory limit, which limits the amount of memory which a single process may occupy at once. (On systems with virtual memory, a user's memory limits apply to their *virtual* memory usage, not their real memory usage.)

CPU Scheduling

The precursor of *Limits* is a similar system which worked in conjunction with the *Share* scheduler, developed at the University of Sydney and the University of New South Wales [9]. *Limits* incorporates the *Share* scheduler as its means for managing the renewable CPU resource.

The *Share* scheduler solves most of the shortcomings of standard UNIX schedulers and other traditional scheduling schemes. Most importantly, *Share* is a *user-level*, not a *process-level* scheduler. Users cannot increase their share of the CPU simply by running more processes concurrently. The *Share* scheduler is hierarchical, allowing division of CPU entitlement at a high level between groups, and further subdivision within such groups.

Share operates by making a user's current share of the CPU proportional to their given entitlement and inversely proportional to their usage of the CPU. Each user's usage value increases as their processes run, and is decayed exponentially to keep it within the bounds of the system's numerical representation. *Share* can be configured to be a *long-term* scheduler by having it decay usage values slowly. One of the advantages of *Share* is that, unlike fixed-budget schedulers, it will not allow the CPU to become idle even if all active users have accumulated extremely large usages. That is, provided that there are always processes to be run, *Share* will never waste CPU time.

Share represents the CPU resource with what is effectively a sophisticated usage and decay model having no limit on usage. The consequence of accumulating a relatively large usage is slow execution of processes, resulting in poor response times. A system administrator may arrange to prevent users from logging in if their effective share of the CPU is so small as to prevent them from accomplishing anything useful. This is essentially the same as adding a universal limit to *Share*'s usage and decay model.

Disk Limits

Disk space is a fixed resource, and is managed accordingly using a usage and limit model. There are, however, *two* disk limits, called the *soft* and *hard* disk limits. The hard disk limit defines a user's absolute maximum disk usage without exception. The soft disk limit defines a user's maximum allowed disk usage while they are not logged in. If a user logs in with a disk usage in excess of their soft limit then they accrue a *warning*. If too many warnings are accrued then the user is denied login, and must report to their respective administrator to have their disk usage reduced or soft disk limit increased. Logging in with a disk usage below the soft limit will remove all accrued warnings.

The soft disk limit should be thought of as the real disk limit, and the hard disk limit as the permitted overshoot. A soft and hard disk limit scheme is necessary to accommodate the typical disk usage habits of most users. In general, disk space is used for two purposes: as long-term storage for data and as temporary working storage. Many programs create potentially large temporary files in the course of their execution, which are automatically removed upon completion. In particular, if a user wishes to compress their data files to reduce their disk usage, they require sufficient temporary storage to perform the compression.

Decentralised Administration

Since *Limits* employs a multi-level, hierarchical group structure for organising users, it is important that the lines of administrative power within that structure be modelled on the lines of authority in a typical organisation. This essentially implies that any user who has authority over a group should be given the means for creating and removing users and controlling resource allocations within only that group.

Limits achieves this by allowing any user to be granted power to create and remove users and control resource allocations within a sub-hierarchy of the entire user hierarchy. That user may also grant those powers to any users belonging to the sub-hierarchy, so that it is possible to have many user administrators operating at many levels. Such user administrators will have smaller spheres of influence the deeper they are situated in the hierarchy.

It is tempting to consider such user administrators as being 'sub-superusers' over their respective sub-hierarchies. This is not quite correct, however, as they are not granted exemption from file permission checking nor the ability to invoke any function of the operating system. This has the advantage of allowing users to keep their files secure from their administrators.

ADDING *LIMITS* TO UNIX

Adding *Limits* to the UNIX operating system involves additions to the kernel and a few modifications to existing programs. There are also some new programs provided specifically for using and maintaining the *Limits* system.

Implementing a system with as wide a scope as *Limits* requires a firm understanding of the designs of many versions of UNIX and their philosophies. To avoid compromising the fundamental design of any UNIX systems, the implementation goals of *Limits* are:

- to require as little modification of UNIX source code as possible;
- to place as much functionality as possible in user-mode programs, in other words, to keep kernel modifications as minimal and as general as possible;
- to involve no modifications to critical programs, in particular the *init* daemon;
- to introduce as little executional overhead as possible to the programs affected by *Limits*;
- to conform with the Posix P1003 standard.

A Per-User Data Structure

The primary obstacle in adding any kind of resource control or distributed administration to UNIX is its lack of a per-user data structure. On a typical UNIX system the only information associated with a user is contained in the password file, the format of which is known by many programs and cannot therefore be altered.

Limits introduces a fundamental change to UNIX: a per-user structure called a "*Limits Node*", or *lnode*. The task of adding full support for the *lnode* structure to UNIX is the most significant aspect of adding *Limits* to UNIX.

Lnodes are stored in a central file and read into memory by the kernel whenever needed. The kernel keeps a cache of *lnodes* to minimise the effect of *Limits* on system performance. In a normal UNIX system each user is defined by a line in the *password file* and is addressed by a unique *login name*. Each user possesses a numerical *User ID* which is used to identify the owner of files and processes. *Limits* uses the UID number as the index into the *lnode* file.

Attributes and Flags

Limits operates by associating with each user a set of *attributes* and *flags*. An *attribute* is a numerical variable which may represent such things as usages, limits, elapsed times, accrued charges, etc. A *flag* is a two-state variable which may be *set* or *clear*, or made to *inherit* its real value from another flag. The attribute and flag values of each user are stored in the user's *lnode*.

Each attribute and flag is identified by a unique name and is declared in the *Limits configuration file*. There are many direct references to standard attributes and flags in the kernel, so their declarations are not amenable to alteration.

As an illustration of the function of attributes, consider the specific attributes called `disk.usage`, `disk.hardlimit` and `disk.accrue`. Whenever a user consumes disk blocks by creating or expanding a file, the number of blocks consumed is added to the `disk.usage` attribute. (This is achieved by small code changes in the relevant kernel routines and introduces a negligible performance overhead.) If the new value of `disk.usage` will exceed the value in `disk.hardlimit` then the consumption is prevented and the user will receive a message to that effect. The value in `disk.usage` is continually accumulated over time in the `disk.accrue` attribute.

By assigning appropriate values to all users' `disk.hardlimit` attributes, a system administrator can suppress excessive disk consumption. If an administration wishes to bill users for their use of disk space, then invoices can be calculated periodically from the values accumulated in the `disk.accrue` attributes. Finally, by taking regular "snapshots" of the `disk.usage` attributes of all or some users, system administrators can obtain daily disk usage profiles which can help them devise measures to cope with periods of high demand.

As an illustration of the application of flags, consider the flag called `flag.nologin`. The *login* program tests the value of this flag whenever it is invoked. If the flag evaluates to *set* then the user is prohibited from logging in and a message is printed explaining that they have been denied access to the system. A system administrator may use this flag to control the permitted login periods of any group of users, and to keep system security tighter by disallowing logins for old, unused user accounts.

A summary of all the standard attributes and flags appears towards the end of this section.

Hierarchy

The users of conventional UNIX systems do not have any particular relation to each other; it is always possible to remove any user from the system without affecting any other user. *Limits* changes this by imposing its own structure on all users: each user occupies a node in a system-wide, hierarchical *scheduling tree*. This is achieved by having in each *lnode* an attribute which identifies its parent *lnode*. The root of the tree is, naturally, the superuser `root`.

A user in the sub tree below user A is said to be a member of the *scheduling group* A, of which user A is the *group header*. A user's immediately enclosing scheduling group is called their *closest* scheduling group, and its group header is the user's parent. A user is therefore considered to be a member of many hierarchically-enclosing scheduling groups, only one of which is its closest, the outermost one always `root`.

Usages and Limits

A fixed resource which may be controlled by *Limits* is generally represented as two attributes: a *usage* and a *limit*. For many resources there is also an *accrued* usage attribute which is not used for control purposes but for billing or accounting if desired.

Whenever a user consumes some of a resource, the extra amount is added to their *usage* value for that resource. The same amount is also added to the usage value of their parent user, and all ascending parent users up to `root`. If any of these additions will exceed the corresponding *limit* value at any stage then the consumption is not allowed. For some resources users will be granted only enough to allow them to reach their nearest limit, whereas for other resources nothing will be granted at all. By convention, a *limit* value of zero indicates that no limit is applied.

Notification

Whenever a user's process reaches either a disk, memory or process limit, it encounters a situation which is identical to reaching the physical limit of the system. This means that programs which have been designed to cope with such situations will naturally cope also with the imposition of a user's limits. Unfortunately, this also means that it will not always be immediately obvious to a user when they have reached a limit.

Limits overcomes this problem by notifying users directly whenever one of their limits is reached. This is accomplished by having the kernel and all modified user-mode processes generate a *notification message* whenever they enforce a limit. The *Limits* daemon process receives all such messages and invokes a delivery script to send each message to its respective user in whatever way is appropriate. This usually involves simply printing a textual message directly on the user's terminal.

Privileges

The administrative privileges that users have are controlled using flags. The flag called `flag.uselim` may be set to allow a user to alter the attribute and flag values of any other user on the system. The flag called `flag.admin` is similar to `flag.uselim`, except that it will grant a user the ability to alter attributes and flags only within the scheduling group of which they are the header. These flags are also tested by the *newuser* program to determine permission for creating or deleting users.

A number of other flags exist which allow normal UNIX operations to be denied to users conditionally. These flags are necessary to allow enforcement of a higher level of security than UNIX is normally able to provide.

A flag may be caused to inherit its real value from the user's parent. This allows the flag settings of entire groups of users to be controlled from a single flag.

Kernel Modifications

In order to implement *Limits* it is necessary to make the following alterations and additions to the UNIX kernel:

- a new system call, `limits()`, which provides a complete interface to all the special functions of the *Limits* system;
- an array to hold the *Inode* cache and code to read and write *Inodes* on demand to and from disk;
- modifications at all points where disk blocks, process table entries or system memory pages are allocated to a user;
- a system call for performing indivisible test-and-set type alterations to any attribute or flag.

Although there are many points in the kernel source code which require alteration, all such alterations are usually straightforward.

Programs

Installing *Limits* involves trivial modifications to a few existing programs such as *login* and *cron*. *Limits* is equipped with several programs of its own which may be used to create, interrogate, modify, copy and remove *Inodes*. There is a *Limits* daemon program which provides the kernel and users of the *Limits* system call with user-mode services such as notification, login/logout monitoring and terminal connect-time measurement. There are also a handful of assorted programs which produce useful dynamic displays and provide global control of the *Limits* system.

A special program, *newuser*, allows any sufficiently privileged user to create, alter and remove users' accounts. If a system already has its own user account control program, then it is straightforward to modify it to conform to the permission flag checking done by *newuser*.

Customisation

The names of all attributes and flags are defined in the *Limits* configuration file which may be altered or expanded by the system administrator at any time. A large area of each *Inode* is not used explicitly by the kernel, and is available for the system administrator to define additional attributes and flags. The *Limits* package includes programs and a library of functions for reading and altering the value of any attribute or flag. These may be used by the system administrator to augment existing resource control software to make use of *Limits* capabilities.

The names and meanings of the built-in attributes and flags are:

`lastused` The time when the user was most recently logged in.
`sgroup` The UID of the closest scheduling group header.
`shares` The number of CPU shares relative to peers.
`myshares` The number of CPU shares relative to children.
`usage` The current *Share* usage value — used to calculate current CPU entitlement.
`charge` The current accrued CPU usage.
`process.limit` Maximum allowed number of concurrent processes.
`disk.usage` Current hard disk consumption.
`disk.hardlimit` Hard disk limit.
`disk.softlimit` Soft disk limit.
`disk.accrue` Accrued disk usage.
`memory.limit` Maximum allowed allocation of virtual memory.
`memory.proclimit` Maximum allowed allocation of virtual memory per process.
`memory.accrue` Accrued virtual memory usage.
`rss.limit` Maximum allowed allocation of system memory.
`rss.proclimit` Maximum allowed allocation of system memory per process.
`flag.admin` If set, grants a user the privilege of altering the attribute and flag values of any member of their scheduling group except themself.
`flag.uselim` If set, grants a user the privilege of altering the attribute and flag values of any user, including themself.
`flag.onelogin` If set, prevents a user from logging in at more then one terminal at a time.
`flag.nologin` If set, denies a user the ability to log in.
`flag.asynckill` If set when a user logs out then any asynchronous processes associated with that login will be terminated unconditionally.
`flag.asyncnice` If set when a user logs out then any asynchronous processes associated with that login will be forced to run at the lowest possible priority.
`flag.chown` If set, allows a user to use the `chown` command

to change the ownership of their files.

`flag.umask` If set, allows a user to alter the permissions of their files without being always subject to their umask.

Typical 'custom' attributes and flags that might be added by an administrator and implemented by modifications to *login* and other relevant programs are:

`warnings` The number of times a user has logged in with a disk usage in excess of their soft disk limit. If a user logs in with a disk limit below their soft limit then this attribute is reset to zero.

`warnlim` The maximum allowed number of accrued warnings before login is denied.

`passwdexpire` The next time at which the user will be requested to choose a new password.

`flag.usenet` If set, grants the privilege of using the network resources such as mail, news, file transfer and so forth.

The most likely customisations to be required are the addition of attributes for controlling resources such as terminal connect time and printer pages. *Limits* incorporates a generic device model called a *device category*, which may be used to deal with many different types of device.

The declaration of a device category in the *Limits* configuration file implicitly defines a standard set of attributes which pertain to all devices which belong to that category. These attributes include a usage value, a limit, and parameters which determine the usage decay characteristics. *Limits* provides library functions and programs designed to manipulate the standard attributes of any device category. A system administrator may modify existing resource control software to invoke these functions and programs at appropriate points.

ADMINISTERING A UNIX SYSTEM WITH *LIMITS*

The basic features of *Limits*, resource control and decentralisation of administration, have been broadly outlined in the previous section. As well as providing these fundamental capabilities, *Limits* provides a solid platform for development of individual solutions to solve a wider range of administrative problems.

Scenario: shared ownership

A pair of educational institutions and a private research institution collectively purchase a powerful, expensive supercomputer. They wish to ensure that the resources of the machine are divided amongst them exactly in proportion to their respective financial investments in it.

This can be achieved using *Limits* by creating three group header users to represent the three institutions, and set up the *shares* and *limit* fields accordingly. Each user will be a member of one of these three scheduling groups according to which institution they belong. It would be the responsibility of the administration of each institution to formulate policy and implement it by creating a scheduling tree within their own scheduling group.

Scenario: special peripherals

A faculty acquires some special graphics terminals for use in several computer aided design courses, and wishes to ensure that they are shared equitably between its departments on a weekly basis. This can be easily accomplished by declaring a new device category and using it to accrue connect time on the workstations. Each department would be represented by its own scheduling group, so each scheduling group header could be given a weekly limit and a decay which reduced usage to zero at the start of each week.

Scenario: `/tmp` cleaning

`/tmp` is a widely-used, publicly accessible directory in which most application programs create and keep temporary working files. Programs occasionally neglect to remove their temporary files from `/tmp`, causing a gradual buildup of unused, leftover files. This problem is commonly solved by running a *tmp-clean* program whenever the system is booted, which removes all files from `/tmp`. On a large system which is booted infrequently, this may not be sufficient to prevent the filesystem containing `/tmp` from becoming full.

The *Limits* daemon may be altered by the system administrator to invoke a shell script upon detecting that a user has just logged out. The shell script could execute commands which remove all the user's files from `/tmp`. If necessary, the administrator could create a new flag called `flag.notmpclean` which, if set, would cause the script not to remove any files.

Scenario: reasons for nologin

A privileged user is able to set the `flag.nologin` flag of any user within their sphere of influence in order to prevent that user from logging in. Any members of that user's scheduling group who inherit their `flag.nologin` flag will also be prevented from logging in. Users so affected may wish to be informed of the reason for denial of login whenever their attempts to log in fail.

The system administrator could comply with such a request by creating flags to represent each possible reason for denial of login. Some examples are:

`flag.nologin.afterhours` This would be set if the `flag.nologin` flag is used to prevent the user from logging in out of normal working hours.

`flag.nologin.owing` This would be set if the user has been denied login for neglecting to pay their bills.

`flag.nologin.antisocial` This would be set if an administrator has barred the user from the system due to antisocial behaviour on the part of the user.

The *login* program could be modified by the system administrator to print a simple 'reason' message whenever login is denied, based on the settings of these flags.

FUTURE ENHANCEMENTS TO *LIMITS*

The current form of *Limits* provides a basis for development of a more comprehensive administration system. This section briefly covers the main areas in which further development is being considered.

Billing

Many UNIX sites currently employ a variety of accounting and database packages to invoice users for services. The *Limits* system maintains accrued usages for most resources and it is only natural that administrators will want to incorporate these figures into their own billing software. Also, administrators of systems without any form of accounting and invoicing software may wish to instigate some form of simple billing once *Limits* is installed.

A simple program may be written which scans any set of users, indivisibly setting an accrued usage value to zero while retrieving the old value. This program, in conjunction with existing UNIX filters, may be used to prepare reports of accrued usage values in a format suitable for input to most accounting and database packages. For sites without any accounting software, existing UNIX tools may be used to construct a simple billing system which maintains account balances and produces invoices.

In a similar fashion, the values of any set of attributes or flags may be entered into a database system. This would prove useful for administrations who wished to analyse user behaviour or resource requirements.

Network Extensions

Network file servers are commonly used to let a file system be accessed by processes on other systems in such a way that the intervening network is not apparent. If a host system permits its clients to create and write files using its network file servers then *Limits* may be used on the host system to control the disk usage of remote users. Alternatively, if a client system has *Limits* installed but the host system does not, then the client may enforce its local disk limits on all file server transactions. This would create an illusion of disk limits acting on the server.

A difficult situation arises when several similar computers, all with *Limits* installed, are closely networked and mutually support network access to each others' file systems. By keeping the systems' password files identical and mounting the network file systems appropriately, the systems may be made to resemble a single system as far as most users are concerned. In order to perpetrate this illusion, a central file of *Inodes* would have to replace the individual systems' files. Also, a form of *Inode* cache coherence algorithm would have to be implemented across the network between the separate kernels.

File Permissions

Each UNIX file has three sets of access permissions which respectively apply to the owner of the file, to users who belong to the file's *permission group*, and to all other users. Different types of UNIX systems provide different ways of specifying file permission groups and their members. The addition of *Limits* to UNIX introduces a hierarchical user group structure which, although completely unrelated to file permission groups, is ideal for exploitation in file permission checking.

A scheme has been proposed in which each file's *permission group* is defined by specifying a user who is the *group owner* of the file. All members of the scheduling group which is headed by the group owner of a file are subject to the file's group permissions. If file permission groups which contradict the scheduling group hierarchy are required, then the system's original group mechanism may still be invoked.

Memory Scheduler

Limits currently treats virtual memory as a non-renewable resource, and manages it with a usage and limit model accordingly. This is sufficient for system administrators who need only prevent excessive memory consumption by some users, but does not contribute towards optimal utilisation of virtual memory and working set memory.

To illustrate this, consider the case of a user who has accumulated a very large CPU usage and consequently has an extremely low effective share of the CPU. It is wasteful of memory to let any of the user's processes occupy a large working set during long periods between being scheduled. It is also wasteful of virtual memory bandwidth to page out the user's processes and page them in whenever they are scheduled. Ideally, the user's processes should remain paged out until the user has attained a large enough effective share to justify paging them in and scheduling them frequently for a significant amount of time.

This illustration shows that it may be worthwhile to extend the *Share* scheduler into the area of memory resource control. A simple *memory scheduler* could be constructed by incorporating appropriate variables from the *Share* calculations into existing virtual memory paging algorithms.

Resource Costs

At present the *Limits* system uses models in which each resource is assumed to have a uniform cost for usage. In reality, however, resources are typically composed of a variety of physical components which have different characteristics such as capacity, speed and price. An administration may wish to charge users different amounts for the different types of each resource, to encourage unbiased consumption.

Changes can be made to the *Limits* kernel modifications to allow a system administrator to specify different costs for different types of resources. In particular, *Limits* can be made to support an individual disk usage cost per file system, and individual memory usage costs for different types of system memory such as fast, slow, shared etc. *Limits* already incorporates a method for specifying different costs for device usage.

CONCLUSION

Limits has been successfully implemented and exercised on an in-house UNIX system at Softway. It has proven to be effective, straightforward and as unobtrusive to users as originally intended. Although the fundamental principle of *Limits* is foreign to the original design philosophy of UNIX, we feel that *Limits* forms a natural and valuable addition to the UNIX environment.

Limits is a powerful administrative tool which allows an administration to implement sophisticated policies easily and effectively. *Limits*, in conjunction with *Share*, has the potential to evolve into the future standard UNIX resource administration system.

REFERENCES

1. Kay, J., and Lauder, P. A fair share scheduler. *Communications of the ACM 31*, 1 (Jan 1988), 44–55.
2. Hume, A. A Share scheduler for UNIX. AUUG Newsletter, Australian UNIX Users' Group, Sydney, Australia, 1979.
3. Kay, J., Lauder, P., Maltby, C., and Tollasep, S. The Share charging and scheduling system. Technical Report 174, Basser Department of Computer Science, University of Sydney, Australia, May 1982.
4. Bach, M.J. *The Design of the UNIX Operating System.* Prentice-Hall, Englewood Cliffs, N.J., 1986.
5. Richie, D.M., and Thompson, K. The UNIX timesharing system. *Bell Systems Technical Journal 57*, 6 (July–Aug 1978), 1905–1929.
6. McKell, L.J., Hansen, J.V., and Heitger, L.E. Charging for computing resources. *ACM Computing Survey 11*, 2 (June 1979), 105–120.
7. Nielsen, N.R. The Allocation of computing resources — Is pricing the answer? *Communications of the ACM 13*, 8 (Aug 1970), 467–474.
8. Kleijnen, A.J.V. Principles of computer charging in a university-type organization. *Communications of the ACM 26*, 11 (Nov 1983), 926–932.
9. Lions, J. Experiences with the UNIX Time-sharing System. *Software — Practice and Experience 9*, (1979), 701–709.